Installing Libraries

```
!pip install plotly kaleido streamlit pyngrok --quiet

import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from collections import Counter
import re
import warnings
warnings.filterwarnings('ignore')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.0/69.0 kB 1.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.2/9.2 MB 26.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.6/52.6 kB 2.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 37.8 MB/s eta 0:00:00
```

```
# Download the compressed .gz file
!wget -q --show-progress "https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz"
print("Downloaded successfully")
```

```
en.openfoodfacts.or 100%[===================>]   1.18G  12.7MB/s    in 98s
Downloaded successfully
```

Story1 - Data Ingestion & Clean Up

```
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

COLS = [
    "product_name",
    "categories_tags",
    "ingredients_text",
    "sugars_100g",
    "proteins_100g",
    "fat_100g",
    "fiber_100g",
    "energy_100g"
]

print("Loading first 500,000 rows...")

df_raw = pd.read_csv(
    "en.openfoodfacts.org.products.csv.gz",
    sep="\t",
    usecols=COLS,
    nrows=500_000,
    low_memory=False,
    on_bad_lines="skip",
    compression="gzip"
)

print(f"Loaded: {df_raw.shape[0]:,} rows × {df_raw.shape[1]} columns")
df_raw.head(3)
```

```
Loading first 500,000 rows...
Loaded: 500,000 rows × 8 columns
```

	product_name	categories_tags	ingredients_text	energy_100g	fat_100g	sugars_100g	fiber_100g	proteins_100g
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	M&M white	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...	NaN	NaN	NaN	NaN	NaN

```
# Data Cleaning

df = df_raw.copy()
# Drop rows missing the three critical fields
df.dropna(subset=["product_name", "sugars_100g", "proteins_100g"], inplace=True)
print(f"After dropping nulls        : {len(df):,} rows")
```

```
# Remove biologically impossible values (nutrients are per 100g, so max is 100)
for col in ["sugars_100g", "proteins_100g", "fat_100g"]:
    df = df[(df[col] >= 0) & (df[col] <= 100)]
print(f"After removing outliers     : {len(df):,} rows")

# Filter energy to realistic range
df = df[(df["energy_100g"] >= 0) & (df["energy_100g"] <= 900)]
print(f"After energy range filter   : {len(df):,} rows")

# Drop duplicate product names
df.drop_duplicates(subset="product_name", keep="first", inplace=True)
df.reset_index(drop=True, inplace=True)
print(f"After deduplication         : {len(df):,} rows")

print("\n Cleaned DataFrame ready!")
df.describe()
```

```
After dropping nulls        : 104,599 rows
After removing outliers     : 104,209 rows
After energy range filter   : 43,550 rows
After deduplication         : 35,059 rows

 Cleaned DataFrame ready!
```

	energy_100g	fat_100g	sugars_100g	fiber_100g	proteins_100g
count	35059.000000	35059.000000	35059.000000	24074.000000	35059.000000
mean	392.333658	3.079312	5.911713	1.305492	4.811836
std	250.652073	4.189846	7.612860	3.001325	6.007131
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	192.928571	0.000000	0.600000	0.000000	0.500000
50%	364.400000	1.200000	3.200000	0.700000	2.666700
75%	586.695402	4.878049	9.200000	1.700000	6.701955
max	900.000000	85.000000	100.000000	89.000000	50.000000

Story2 - The Category Wrangler

```
#Parse categories_tags column
#Assign a Primary Category using keyword logic
#At least 5 distinct high-level buckets

CATEGORY_MAP = {
    "Bars & Snack Bars":  ["bar", "granola-bar", "protein-bar", "cereal-bar"],
    "Chips & Crisps":     ["chip", "crisp", "popcorn", "pretzel", "puffed"],
    "Cookies & Biscuits": ["cookie", "biscuit", "wafer", "cracker"],
    "Chocolate & Candy":  ["chocolate", "candy", "confectionery", "sweet", "gummy"],
    "Nuts & Seeds":       ["nut", "seed", "almond", "peanut", "cashew", "pistachio"],
    "Dairy Snacks":       ["yogurt", "yoghurt", "cheese", "dairy"],
    "Fruit Snacks":       ["fruit", "dried-fruit", "raisin", "date"],
    "Other Snacks":       [], }

def assign_category(tags_str):
    if pd.isna(tags_str):
        return "Other Snacks"
    tags = tags_str.lower()
    for category, keywords in CATEGORY_MAP.items():
        if keywords and any(kw in tags for kw in keywords):
            return category
    return "Other Snacks"

df["primary_category"] = df["categories_tags"].apply(assign_category)

print("Category distribution (must have at least 5 non-empty buckets):")
dist = df["primary_category"].value_counts()
print(dist)
print(f"\nDistinct categories: {dist[dist > 0].count()} ")
```

```
Category distribution (must have at least 5 non-empty buckets):
primary_category
Other Snacks          27551
Dairy Snacks           3468
Fruit Snacks           2063
Chocolate & Candy      1044
Nuts & Seeds            477
Bars & Snack Bars       185
Chips & Crisps          147
Cookies & Biscuits      124
Name: count, dtype: int64
```

```
Distinct categories: 8
```

Candidate's Choice:The Opportunity Score

```
# WHY I ADDED THIS:
#   Charts show patterns but clients need decisions. Instead of asking the
#   R&D team to interpret multiple charts, this feature collapses every health
#   and competition signal into a single 0-100 score per category.
#
#   Score weights (justified by business logic):
#     • Protein (35%) — #1 consumer health trend in snacking (Nielsen 2023)
#     • Low Sugar (30%) — the core brief from the client
#     • Fiber  (20%) — secondary health signal, increasingly demanded
#     • Low Competition (15%) — fewer products = easier shelf placement & margin
#
#   This is directly actionable: the R&D team can take the #1 ranked category
#   straight into product development without further analysis.

def compute_opportunity_score(group):
    total        = len(group)
    high_protein = (group["proteins_100g"] > 10).mean()
    low_sugar    = (group["sugars_100g"]   <  5).mean()
    # Fill fiber nulls with 0 for scoring — missing = no fiber benefit
    fiber_vals   = group["fiber_100g"].fillna(0)
    high_fiber   = (fiber_vals > 3).mean()
    competition  = 1 / (1 + total / 500)   # inverse crowding

    score = (
        high_protein * 35 +
        low_sugar    * 30 +
        high_fiber   * 20 +
        competition  * 15
    )
    return round(score, 2)

score_list = []
for cat, group in df.groupby("primary_category"):
    score_list.append({
        "primary_category": cat,
        "opportunity_score": compute_opportunity_score(group)
    })
scores = pd.DataFrame(score_list)

# Supporting metrics
cat_stats = df.groupby("primary_category").agg(
    total_products   = ("product_name",  "count"),
    avg_protein      = ("proteins_100g", "mean"),
    avg_sugar        = ("sugars_100g",   "mean"),
    avg_fiber        = ("fiber_100g",    "mean"),
).reset_index()

# Blue ocean count — products with high protein AND low sugar
blue_counts = (
    df[df["proteins_100g"] > 10]
    .groupby("primary_category")
    .apply(lambda g: (g["sugars_100g"] < 5).sum())
    .reset_index()
)
blue_counts.columns = ["primary_category", "blue_ocean_count"]

opportunity_df = (
    scores
    .merge(cat_stats,    on="primary_category")
    .merge(blue_counts,  on="primary_category", how="left")
    .sort_values("opportunity_score", ascending=False)
    .reset_index(drop=True)
)
opportunity_df["blue_ocean_count"] = opportunity_df["blue_ocean_count"].fillna(0).astype(int)
opportunity_df["rank"] = opportunity_df.index + 1

print("Market Opportunity Scores:")
print(opportunity_df[[
    "rank","primary_category","opportunity_score",
    "avg_protein","avg_sugar","total_products","blue_ocean_count"
]].to_string(index=False))
```

```
Market Opportunity Scores:
 rank   primary_category  opportunity_score  avg_protein  avg_sugar  total_products  blue_ocean_count
    1     Chips & Crisps              42.88     2.649209   1.917166             147                 4
    2       Nuts & Seeds              39.68     3.728766   2.495391             477                16
```

```
    3       Other Snacks              28.02     5.165088   5.245521           27551              4539
    4  Bars & Snack Bars              26.98     3.191265  17.761356             185                21
    5 Cookies & Biscuits              26.62     3.631246  10.861146             124                 7
    6       Fruit Snacks              20.55     1.345295   6.753079            2063                12
    7  Chocolate & Candy              17.03     0.741543  11.571749            1044                 4
    8       Dairy Snacks              12.48     5.662225   8.829909            3468               199
```

Save Parquet Files

```
# Saving a cleaned data to streamlit
df.to_parquet("cleaned_food_data.parquet", index=False)
print(f"Saved {len(df):,} rows to cleaned_food_data.parquet")
```

```
Saved 35,059 rows to cleaned_food_data.parquet
```

The Nutrient Matrix Visualization ,The Recommendation and the Hidden Gem

```
APP_CODE = '''
import streamlit as st
import pandas as pd
import plotly.express as px
from collections import Counter

st.set_page_config(
    page_title="Sugar Trap — Market Gap Analysis",
    layout="wide"
)

# Load data
@st.cache_data
def load_data():
    df  = pd.read_parquet("cleaned_food_data.parquet")
    opp = pd.read_parquet("opportunity_scores.parquet")
    return df, opp

df, opportunity_df = load_data()

st.sidebar.title("Filters")
st.sidebar.caption("Use these filters to explore the data across all charts.")

all_cats      = sorted(df["primary_category"].unique())
selected_cats = st.sidebar.multiselect(
    "Product Category", all_cats, default=all_cats
)
sugar_max   = st.sidebar.slider("Max sugar (g per 100g)",   0, 100, 100)
protein_min = st.sidebar.slider("Min protein (g per 100g)", 0,  50,   0)

fdf = df[
    df["primary_category"].isin(selected_cats) &
    (df["sugars_100g"]   <= sugar_max) &
    (df["proteins_100g"] >= protein_min)
]

# DASHBOARD HEADER

st.title("Sugar Trap — Snack Market Gap Analysis")
st.markdown(
    "**Client:** Helix CPG Partners  |  "
    "**Dataset:** Open Food Facts (500,000 products)  |  "
    "**Objective:** Find the Blue Ocean in the snack aisle"
)
st.markdown("---")

# KPI ROW — At-a-glance metrics
col1, col2, col3, col4 = st.columns(4)

col1.metric(
    label="Total Products",
    value=f"{len(fdf):,}"
)
col2.metric(
    label="Avg Sugar (g per 100g)",
    value=f"{fdf['sugars_100g'].mean():.1f}g"
)
col3.metric(
    label="Avg Protein (g per 100g)",
    value=f"{fdf['proteins_100g'].mean():.1f}g"
)
blue_ocean = fdf[
```

```
    (fdf["proteins_100g"] > 10) & (fdf["sugars_100g"] < 5)
]
col4.metric(
    label="Blue Ocean Products",
    value=f"{len(blue_ocean):,}",
    help="Products with >10g protein AND <5g sugar per 100g"
)

st.markdown("---")
st.subheader("Story 3 — Nutrient Matrix Visualization")
st.caption(
    "Every dot is one product, plotted by its sugar content (X-axis) vs "
    "protein content (Y-axis). The green shaded zone in the top-left corner "
    "is the Blue Ocean — High Protein + Low Sugar — where very few products "
    "currently exist. This is the biggest gap in the market."
)

plot_df = (
    fdf.sample(min(5000, len(fdf)), random_state=42)
    if len(fdf) > 5000 else fdf
)

fig_scatter = px.scatter(
    plot_df,
    x="sugars_100g",
    y="proteins_100g",
    color="primary_category",
    hover_name="product_name",
    opacity=0.55,
    labels={
        "sugars_100g":      "Sugar (g per 100g)",
        "proteins_100g":    "Protein (g per 100g)",
        "primary_category": "Category",
    },
    title="Nutrient Matrix Visualization — Sugar vs Protein by Category",
    height=540,
)

max_prot = fdf["proteins_100g"].max()

# Highlight the Blue Ocean quadrant
fig_scatter.add_shape(
    type="rect",
    x0=0, x1=5, y0=10, y1=max_prot + 2,
    fillcolor="rgba(30,180,120,0.12)",
    line=dict(color="rgba(30,180,120,0.6)", width=1.5)
)
fig_scatter.add_annotation(
    x=2.5, y=max_prot + 1,
    text="🟢 Blue Ocean: High Protein + Low Sugar (Empty Quadrant)",
    showarrow=False,
    font=dict(size=11, color="green")
)

# Quadrant divider lines
fig_scatter.add_vline(
    x=5, line_dash="dash", line_color="gray",
    annotation_text="Sugar threshold (5g)",
    annotation_position="top right"
)
fig_scatter.add_hline(
    y=10, line_dash="dash", line_color="gray",
    annotation_text="Protein threshold (10g)",
    annotation_position="top right"
)
fig_scatter.update_layout(legend_title_text="Category")
st.plotly_chart(fig_scatter, use_container_width=True)

st.markdown("---")

# CANDIDATE\'S CHOICE: MARKET OPPORTUNITY SCORE LEADERBOARD
#
# As a Creative Analyst, I added a weighted scoring model that converts
# every health and competition signal into a single 0-100 score per
# category — so the R&D team gets a clear ranked answer, not just charts.
#
# Score = Protein signal (35%) + Low Sugar (30%) + Fiber (20%)
#         + Low Competition (15%)
st.subheader("Candidate's Choice — Market Opportunity Score Leaderboard")
st.info(
    "**Why I built this:** Charts show patterns but clients need decisions. "
    "This leaderboard converts every health and competition signal into a "
```

```
    "single 0–100 score per category, so the R&D team can immediately see "
    "WHERE to launch without interpreting multiple charts.\\n\\n"
    "**Score formula:** "
    "Protein signal (35%) + Low Sugar signal (30%) + "
    "Fiber signal (20%) + Low Competition (15%)"
)

fopp = opportunity_df[
    opportunity_df["primary_category"].isin(selected_cats)
].copy()

fig_opp = px.bar(
    fopp.sort_values("opportunity_score"),
    x="opportunity_score",
    y="primary_category",
    orientation="h",
    color="opportunity_score",
    color_continuous_scale=["#d73027", "#fee08b", "#1a9850"],
    text="opportunity_score",
    hover_data={
        "total_products":   True,
        "avg_protein":      ":.1f",
        "avg_sugar":        ":.1f",
        "blue_ocean_count": True,
    },
    labels={
        "opportunity_score":  "Opportunity Score (0–100)",
        "primary_category":   "Category",
        "total_products":     "Total products",
        "avg_protein":        "Avg protein (g)",
        "avg_sugar":          "Avg sugar (g)",
        "blue_ocean_count":   "Blue Ocean products",
    },
    title="Market Opportunity Score by Category — Higher = Bigger Blue Ocean",
    height=420,
)
fig_opp.update_traces(texttemplate="%{text:.1f}", textposition="outside")
fig_opp.update_layout(coloraxis_showscale=False, yaxis_title="")
st.plotly_chart(fig_opp, use_container_width=True)

# Score breakdown table
st.markdown("##### Full score breakdown by category")
display_df = fopp[[
    "rank", "primary_category", "opportunity_score",
    "avg_protein", "avg_sugar", "avg_fiber",
    "total_products", "blue_ocean_count"
]].rename(columns={
    "rank":             "Rank",
    "primary_category": "Category",
    "opportunity_score":"Score",
    "avg_protein":      "Avg Protein (g)",
    "avg_sugar":        "Avg Sugar (g)",
    "avg_fiber":        "Avg Fiber (g)",
    "total_products":   "Total Products",
    "blue_ocean_count": "Blue Ocean Products",
}).round(2)

st.dataframe(
    display_df.style.background_gradient(subset=["Score"], cmap="RdYlGn"),
    use_container_width=True,
    hide_index=True,
)

if len(fopp) > 0:
    winner = fopp.loc[fopp["opportunity_score"].idxmax()]
    st.success(
        f"🥇 **Top opportunity: {winner[\'primary_category\']}** "
        f"(Score: {winner[\'opportunity_score\']:.1f} / 100) — "
        f"Only {int(winner[\'blue_ocean_count\'])} products currently occupy "
        f"the Blue Ocean zone, with avg protein of {winner[\'avg_protein\']:.1f}g "
        f"and avg sugar of {winner[\'avg_sugar\']:.1f}g per 100g."
    )

st.markdown("---")

#FAT vs FIBER HEALTH HEATMAP
# Additional chart showing the second axis of health beyond Sugar vs Protein
st.subheader("Fat vs Fiber Health Heatmap")
st.caption(
    "Sugar vs Protein alone does not tell the full story. "
    "A product can be low-sugar and high-protein but still be unhealthy "
    "if it is high in fat and zero in fiber. "
```

```
    "This heatmap reveals the second dimension of the Blue Ocean — "
    "the ideal zone is top-left: High Fiber + Low Fat."
)

heat_df = fdf[
    fdf["fiber_100g"].notna() & fdf["fat_100g"].notna()
].copy()
heat_df = heat_df.sample(min(3000, len(heat_df)), random_state=1)

fig_heat = px.density_heatmap(
    heat_df,
    x="fat_100g",
    y="fiber_100g",
    nbinsx=30, nbinsy=30,
    color_continuous_scale="RdYlGn",
    labels={
        "fat_100g":   "Fat (g per 100g)",
        "fiber_100g": "Fiber (g per 100g)",
    },
    title="Fat vs Fiber Density — Where are the truly healthy snacks?",
    height=440,
)
fig_heat.add_annotation(
    x=3,
    y=heat_df["fiber_100g"].max() * 0.85,
    text="🟢 Ideal zone: High Fiber + Low Fat",
    showarrow=False,
    font=dict(size=11, color="darkgreen")
)
st.plotly_chart(fig_heat, use_container_width=True)

st.markdown("---")

# ✅ Key Insight box included on dashboard
#  Sentence: "Based on the data, the biggest market opportunity
#    is in [Category], specifically targeting products with [X]g
#    of protein and less than [Y]g of sugar."
st.subheader("Story 4 — Key Market Insight & Recommendation")

best_cat = (
    df.groupby("primary_category")
    .apply(lambda g: int((
        (g["proteins_100g"] > 10) & (g["sugars_100g"] < 5)
    ).sum()))
    .idxmax()
)
best_grp = df[
    (df["primary_category"] == best_cat) &
    (df["proteins_100g"]    >  10) &
    (df["sugars_100g"]      <   5)
]
avg_prot = best_grp["proteins_100g"].mean()
avg_sug  = best_grp["sugars_100g"].mean()

st.success(
    f"**Based on the data, the biggest market opportunity is in "
    f"*{best_cat}*, specifically targeting products with "
    f"{avg_prot:.0f}g of protein and less than {avg_sug:.0f}g of sugar "
    f"per 100g.**\\n\\n"
    f"Only {len(best_grp):,} products currently occupy this space across "
    f"500,000 products analysed — a clear, underserved gap in the snack aisle "
    f"that represents a significant first-mover advantage for our client."
)

st.markdown("---")

# BONUS STORY: THE HIDDEN GEM
#
# As a Health Conscious Consumer, I want to know which specific
# ingredients are driving the high protein content in the "good"
# products, so that I can replicate this in our new recipe.
#
# Analysed ingredients_text for High Protein cluster
# Extracted Top 3 most common protein sources
st.subheader("Bonus Story — The Hidden Gem: Top Protein Sources")
st.caption(
    "Analysing the ingredients lists of all High-Protein (>10g) / "
    "Low-Sugar (<5g) products to identify which protein sources the "
    "R&D team should use when formulating the new product line."
)

hp_df = df[
```

```
    (df["proteins_100g"] > 10) &
    (df["sugars_100g"]   <  5) &
    df["ingredients_text"].notna()
]

PROTEIN_SOURCES = [
    "whey", "casein", "soy", "pea protein", "egg", "peanut", "almond",
    "milk protein", "hemp", "rice protein", "chickpea", "lentil",
    "quinoa", "sunflower seed", "pumpkin seed",
]

counts = Counter()
for text in hp_df["ingredients_text"].dropna():
    t = text.lower()
    for src in PROTEIN_SOURCES:
        if src in t:
            counts[src] += 1

top10 = counts.most_common(10)

if top10:
    src_df = pd.DataFrame(top10, columns=["Ingredient", "Count"])
    fig_ing = px.bar(
        src_df,
        x="Count",
        y="Ingredient",
        orientation="h",
        color="Count",
        color_continuous_scale="Teal",
        title="Most Common Protein Sources in High-Protein / Low-Sugar Products",
        height=400,
    )
    fig_ing.update_layout(yaxis=dict(autorange="reversed"))
    st.plotly_chart(fig_ing, use_container_width=True)

    top3 = ", ".join([x[0].title() for x in top10[:3]])
    st.info(
        f"**Top 3 protein sources found:** {top3}\\n\\n"
        f"**R&D Recommendation:** Formulate the new product line "
        f"using these ingredients to credibly deliver on the "
        f"high-protein brief while maintaining a clean label."
    )
else:
    st.warning(
        "Not enough ingredient data for the current filter combination. "
        "Try widening your filters in the sidebar."
    )


st.markdown("---")
st.caption(
    "📊 Data source: Open Food Facts (openfoodfacts.org) — "
    "First 500,000 rows | "
    "Analysis: Helix CPG Partners Market Gap Study | "
    "Built with Streamlit & Plotly"
)
'''

with open("app.py", "w") as f:
    f.write(APP_CODE)

print("app.py written successfully")
```

```
app.py written successfully
```

Diagonistic Check

```
import subprocess

# Check if Streamlit is actually running
result = subprocess.run(["pgrep", "-f", "streamlit"], capture_output=True, text=True)
print("Streamlit process IDs:", result.stdout.strip() or "NONE — not running!")

# Check if port 8501 is listening
port_check = subprocess.run(["ss", "-tlnp"], capture_output=True, text=True)
for line in port_check.stdout.split("\n"):
    if "8501" in line:
        print("Port 8501 status:", line)
        break
else:
```

```
    print("Port 8501: NOT listening")

# Check if app.py exists
import os
print("app.py exists:", os.path.exists("app.py"))
```

```
Streamlit process IDs: 23464
Port 8501 status: LISTEN 0      2048         0.0.0.0:8501       0.0.0.0:*    users:(("streamlit",pid=23464,fd=13))
app.py exists: True
```

Dashboard Launching

Installing Libraries

```
!pip install plotly kaleido streamlit pyngrok --quiet

import pandas as pd
import numpy as np
import plotly.express as px
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

print("Ready")
```

```
Ready
```

```
!wget -q --show-progress "https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz"
print("Download complete")
```

```
en.openfoodfacts.or 100%[===================>]   1.18G  16.6MB/s    in 78s
Download complete
```

Data Ingestion

```
COLS = [
    "product_name", "categories_tags", "ingredients_text",
    "sugars_100g", "proteins_100g", "fat_100g",
    "fiber_100g", "energy_100g"
]

print("Loading 500,000 rows...")

df_raw = pd.read_csv(
    "en.openfoodfacts.org.products.csv.gz",
    sep="\t",
    usecols=COLS,
    nrows=500_000,
    low_memory=False,
    on_bad_lines="skip",
    compression="gzip"
)

print(f"Loaded: {df_raw.shape[0]:,} rows × {df_raw.shape[1]} columns")
df_raw.head(3)
```

```
Loading 500,000 rows...
Loaded: 500,000 rows × 8 columns
```

	product_name	categories_tags	ingredients_text	energy_100g	fat_100g	sugars_100g	fiber_100g	proteins_100g
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	M&M white	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...	NaN	NaN	NaN	NaN	NaN

Data Cleaning

```
df = df_raw.copy()

# Remove rows missing critical nutritional fields
df.dropna(subset=["product_name", "sugars_100g", "proteins_100g"], inplace=True)
print(f"After dropping nulls          : {len(df):,} rows")
```

```
# Remove biologically impossible values
for col in ["sugars_100g", "proteins_100g", "fat_100g"]:
    df = df[(df[col] >= 0) & (df[col] <= 100)]
print(f"After removing outliers       : {len(df):,} rows")

# Energy is stored in kJ realistic max is 4,000 kJ per 100g
df = df[df["energy_100g"].isna() | (
    (df["energy_100g"] >= 0) & (df["energy_100g"] <= 4000)
)]
print(f"After energy filter           : {len(df):,} rows")

# Remove duplicate product names
df.drop_duplicates(subset="product_name", keep="first", inplace=True)
df.reset_index(drop=True, inplace=True)
print(f"After deduplication           : {len(df):,} rows")
print(f"\nFinal cleaned shape: {df.shape}")
df.describe().round(2)
```

```
After dropping nulls          : 104,845 rows
After removing outliers       : 104,453 rows
After energy filter           : 104,363 rows
After deduplication           : 81,917 rows

Final cleaned shape: (81917, 8)
```

	energy_100g	fat_100g	sugars_100g	fiber_100g	proteins_100g
count	81910.00	81917.00	81917.00	59357.00	81917.00
mean	1133.23	11.40	13.15	2.98	8.17
std	757.04	14.81	18.61	5.13	9.77
min	0.00	0.00	0.00	0.00	0.00
25%	437.60	0.80	1.12	0.00	2.00
50%	1106.80	5.22	4.69	1.60	6.00
75%	1700.00	17.50	17.01	3.57	10.59
max	3940.93	100.00	100.00	100.00	100.00

Category Wrangler

```
CATEGORY_MAP = {
    "Bars & Snack Bars":  ["bar", "granola-bar", "protein-bar", "cereal-bar"],
    "Chips & Crisps":     ["chip", "crisp", "popcorn", "pretzel", "puffed"],
    "Cookies & Biscuits": ["cookie", "biscuit", "wafer", "cracker"],
    "Chocolate & Candy":  ["chocolate", "candy", "confectionery", "sweet", "gummy"],
    "Nuts & Seeds":       ["nut", "seed", "almond", "peanut", "cashew", "pistachio"],
    "Dairy Snacks":       ["yogurt", "yoghurt", "cheese", "dairy"],
    "Fruit Snacks":       ["fruit", "dried-fruit", "raisin", "date"],
    "Other Snacks":       [],
}

def assign_category(tags_str):
    if pd.isna(tags_str):
        return "Other Snacks"
    tags = tags_str.lower()
    for category, keywords in CATEGORY_MAP.items():
        if keywords and any(kw in tags for kw in keywords):
            return category
    return "Other Snacks"

df["primary_category"] = df["categories_tags"].apply(assign_category)

dist = df["primary_category"].value_counts()
print(f"{len(dist)} categories created:\n")
print(dist.to_string())
```

```
8 categories created:

primary_category
Other Snacks          64718
Dairy Snacks           4652
Chocolate & Candy      3670
Fruit Snacks           2497
Cookies & Biscuits     2257
Nuts & Seeds           1786
Chips & Crisps         1378
Bars & Snack Bars       959
```

Candidate's Choice:Opportunity Score

```
def compute_opportunity_score(group):
    high_protein = (group["proteins_100g"] > 10).mean()
    low_sugar    = (group["sugars_100g"] < 5).mean()
    high_fiber   = (group["fiber_100g"].fillna(0) > 3).mean()
    competition  = 1 / (1 + len(group) / 500)
    return round(
        high_protein * 35 +
        low_sugar    * 30 +
        high_fiber   * 20 +
        competition  * 15, 2
    )

score_list = [
    {"primary_category": cat, "opportunity_score": compute_opportunity_score(grp)}
    for cat, grp in df.groupby("primary_category")
]
scores = pd.DataFrame(score_list)

cat_stats = df.groupby("primary_category").agg(
    total_products=("product_name",  "count"),
    avg_protein   =("proteins_100g", "mean"),
    avg_sugar     =("sugars_100g",   "mean"),
    avg_fiber     =("fiber_100g",    "mean"),
).reset_index()

blue_counts = pd.DataFrame([
    {"primary_category": cat,
     "blue_ocean_count": int(((g["proteins_100g"] > 10) & (g["sugars_100g"] < 5)).sum())}
    for cat, g in df.groupby("primary_category")
])

opportunity_df = (
    scores
    .merge(cat_stats,   on="primary_category")
    .merge(blue_counts, on="primary_category")
    .sort_values("opportunity_score", ascending=False)
    .reset_index(drop=True)
)
opportunity_df["rank"] = opportunity_df.index + 1

print("Opportunity scores computed:")
print(opportunity_df[["rank","primary_category","opportunity_score",
                      "avg_protein","avg_sugar","blue_ocean_count"]].to_string(index=False))
```

```
Opportunity scores computed:
 rank   primary_category  opportunity_score  avg_protein  avg_sugar  blue_ocean_count
    1       Nuts & Seeds              52.13    13.139329   6.200714               495
    2     Chips & Crisps              41.23     6.605675   5.968382                92
    3       Other Snacks              31.24     8.596655  11.740827             13654
    4  Bars & Snack Bars              25.18     8.211702  29.453723                49
    5       Dairy Snacks              22.93     8.865719   7.599528              1021
    6       Fruit Snacks              21.30     2.237518  11.788133                39
    7 Cookies & Biscuits              17.74     6.159952  24.180706               102
    8  Chocolate & Candy              11.40     3.219631  40.959045                47
```

Save Files

```
df.to_parquet("cleaned_food_data.parquet", index=False)
opportunity_df.to_parquet("opportunity_scores.parquet", index=False)
print(f"cleaned_food_data.parquet  — {len(df):,} rows")
print(f"opportunity_scores.parquet — {len(opportunity_df)} categories")
```

```
cleaned_food_data.parquet  — 81,917 rows
opportunity_scores.parquet — 8 categories
```

Nutrient Matrix Visualization,Recommendation and The Hidden Gem

```
APP_CODE = '''
import streamlit as st
import pandas as pd
import plotly.express as px
from collections import Counter

st.set_page_config(
    page_title="Sugar Trap | Market Gap Analysis",
    page_icon="📊",
```

```
    layout="wide"
)

@st.cache_data
def load_data():
    df  = pd.read_parquet("cleaned_food_data.parquet")
    opp = pd.read_parquet("opportunity_scores.parquet")
    return df, opp

df, opportunity_df = load_data()

# ── Sidebar ──────────────────────────────────────────────────────
st.sidebar.image(
    "https://upload.wikimedia.org/wikipedia/commons/thumb/1/13/Helix_nebula_symbol.svg/240px-Helix_nebula_symbol.svg.png",
    width=60
)
st.sidebar.title("Filters")
st.sidebar.markdown("Adjust filters to explore the data across all charts.")
st.sidebar.markdown("---")

all_cats = sorted(df["primary_category"].unique())
selected_cats = st.sidebar.multiselect(
    "Product Category", all_cats, default=all_cats
)
sugar_max   = st.sidebar.slider("Max Sugar (g per 100g)",   0, 100, 100)
protein_min = st.sidebar.slider("Min Protein (g per 100g)", 0,  50,   0)

st.sidebar.markdown("---")
st.sidebar.caption(
    "Data: Open Food Facts | 500,000 products analysed"
)

fdf = df[
    df["primary_category"].isin(selected_cats) &
    (df["sugars_100g"]   <= sugar_max) &
    (df["proteins_100g"] >= protein_min)
]

# Header
st.title("Sugar Trap — Snack Market Gap Analysis")
st.markdown(
    "**Client:** Helix CPG Partners  ·  "
    "**Dataset:** Open Food Facts (500,000 products)  ·  "
    "**Question:** Where is the Blue Ocean in the snack aisle?"
)
st.markdown("---")

#  KPI Row
k1, k2, k3, k4 = st.columns(4)
blue_ocean_products = fdf[
    (fdf["proteins_100g"] > 10) & (fdf["sugars_100g"] < 5)
]
k1.metric("Products Analysed",       f"{len(fdf):,}")
k2.metric("Avg Sugar (g/100g)",      f"{fdf['sugars_100g'].mean():.1f}g")
k3.metric("Avg Protein (g/100g)",    f"{fdf['proteins_100g'].mean():.1f}g")
k4.metric("Blue Ocean Products",     f"{len(blue_ocean_products):,}",
          help="High Protein (>10g) AND Low Sugar (<5g) per 100g")
st.markdown("---")


# STORY 3 — NUTRIENT MATRIX VISUALIZATION
# Scatter plot: Sugar (X) vs Protein (Y), coloured by category.
# The empty top-left quadrant is the Blue Ocean opportunity.

st.subheader("Story 3 — Nutrient Matrix Visualization")
st.caption(
    "Each dot represents one product. The green zone (top-left) is the "
    "Blue Ocean — high protein and low sugar — where almost no products "
    "currently exist. Use the sidebar to filter by category."
)

plot_df = fdf.sample(min(5000, len(fdf)), random_state=42) if len(fdf) > 5000 else fdf

fig_scatter = px.scatter(
    plot_df,
    x="sugars_100g",
    y="proteins_100g",
    color="primary_category",
    hover_name="product_name",
    opacity=0.5,
    labels={
        "sugars_100g":      "Sugar (g per 100g)",
```

```
        "proteins_100g":    "Protein (g per 100g)",
        "primary_category": "Category",
    },
    title="Nutrient Matrix Visualization — Sugar vs Protein by Snack Category",
    height=540,
)

max_prot = float(fdf["proteins_100g"].max())

fig_scatter.add_shape(
    type="rect", x0=0, x1=5, y0=10, y1=max_prot + 2,
    fillcolor="rgba(30,180,120,0.10)",
    line=dict(color="rgba(30,180,120,0.6)", width=1.5)
)
fig_scatter.add_annotation(
    x=2.5, y=max_prot + 1,
    text="Blue Ocean: High Protein + Low Sugar (Empty Quadrant)",
    showarrow=False, font=dict(size=11, color="#1a7a4a")
)
fig_scatter.add_vline(
    x=5, line_dash="dash", line_color="#aaaaaa",
    annotation_text="Sugar cut-off (5g)",
    annotation_position="top right"
)
fig_scatter.add_hline(
    y=10, line_dash="dash", line_color="#aaaaaa",
    annotation_text="Protein cut-off (10g)",
    annotation_position="top right"
)
fig_scatter.update_layout(
    legend_title_text="Category",
    plot_bgcolor="white",
    paper_bgcolor="white",
    font=dict(family="Arial", size=12),
)
st.plotly_chart(fig_scatter, use_container_width=True)
st.markdown("---")

# CANDIDATE'S CHOICE — MARKET OPPORTUNITY SCORE LEADERBOARD
# Why I built this:
# A scatter plot shows where the gap is but a client needs to know
# WHICH category to enter. This scoring model ranks every category
# on a 0-100 scale using four weighted signals, giving the R&D team
# one clear, defensible answer without interpreting multiple charts.
# Weights: Protein (35%) | Low Sugar (30%) | Fiber (20%) | Low Competition (15%)

st.subheader("Candidate's Choice — Market Opportunity Score Leaderboard")
st.info(
    "**Why I built this:** A scatter plot shows where the gap is, but a "
    "client needs to know which category to enter first. This leaderboard "
    "turns four data signals into one ranked score per category — giving "
    "the R&D team a single, defensible answer they can take to the boardroom. "
    "\\n\\n**Score weights:** Protein signal (35%) · Low Sugar signal (30%) · "
    "Fiber signal (20%) · Low Competition (15%)"
)

fopp = opportunity_df[
    opportunity_df["primary_category"].isin(selected_cats)
].copy()

fig_opp = px.bar(
    fopp.sort_values("opportunity_score"),
    x="opportunity_score",
    y="primary_category",
    orientation="h",
    color="opportunity_score",
    color_continuous_scale=["#d73027", "#fee08b", "#1a9850"],
    text="opportunity_score",
    hover_data={
        "total_products":   True,
        "avg_protein":      ":.1f",
        "avg_sugar":        ":.1f",
        "blue_ocean_count": True,
    },
    labels={
        "opportunity_score":  "Opportunity Score (0–100)",
        "primary_category":   "Category",
        "total_products":     "Total Products",
        "avg_protein":        "Avg Protein (g)",
        "avg_sugar":          "Avg Sugar (g)",
        "blue_ocean_count":   "Blue Ocean Products",
    },
```

```
    title="Market Opportunity Score — Which Category Has the Biggest Gap?",
    height=420,
)
fig_opp.update_traces(texttemplate="%{text:.1f}", textposition="outside")
fig_opp.update_layout(
    coloraxis_showscale=False,
    yaxis_title="",
    plot_bgcolor="white",
    paper_bgcolor="white",
    font=dict(family="Arial", size=12),
)
st.plotly_chart(fig_opp, use_container_width=True)

st.markdown("##### Score breakdown by category")
display_df = fopp[[
    "rank", "primary_category", "opportunity_score",
    "avg_protein", "avg_sugar", "avg_fiber",
    "total_products", "blue_ocean_count"
]].rename(columns={
    "rank":             "Rank",
    "primary_category": "Category",
    "opportunity_score":"Score",
    "avg_protein":      "Avg Protein (g)",
    "avg_sugar":        "Avg Sugar (g)",
    "avg_fiber":        "Avg Fiber (g)",
    "total_products":   "Total Products",
    "blue_ocean_count": "Blue Ocean Products",
}).round(2)

st.dataframe(
    display_df.style.background_gradient(subset=["Score"], cmap="RdYlGn"),
    use_container_width=True,
    hide_index=True,
)

if len(fopp) > 0:
    winner = fopp.loc[fopp["opportunity_score"].idxmax()]
    st.success(
        f"**Top opportunity: {winner['primary_category']}** "
        f"(Score: {winner['opportunity_score']:.1f} / 100) — "
        f"Only {int(winner['blue_ocean_count'])} products currently sit in the "
        f"Blue Ocean zone, with an average of {winner['avg_protein']:.1f}g protein "
        f"and {winner['avg_sugar']:.1f}g sugar per 100g."
    )

st.markdown("---")

# FAT vs FIBER HEATMAP
# Supporting chart showing the second health dimension.
# The ideal zone — high fiber, low fat — is equally under-served.
st.subheader("Fat vs Fiber Health Heatmap")
st.caption(
    "Protein and sugar are only part of the picture. This heatmap adds fat "
    "and fiber — the two other critical health signals. The top-left zone "
    "(high fiber, low fat) is where truly clean-label snacks would live. "
    "It is almost entirely empty."
)

heat_df = fdf[fdf["fiber_100g"].notna() & fdf["fat_100g"].notna()].copy()
heat_df  = heat_df.sample(min(3000, len(heat_df)), random_state=1)

fig_heat = px.density_heatmap(
    heat_df,
    x="fat_100g",
    y="fiber_100g",
    nbinsx=30, nbinsy=30,
    color_continuous_scale="RdYlGn",
    labels={
        "fat_100g":   "Fat (g per 100g)",
        "fiber_100g": "Fiber (g per 100g)",
    },
    title="Fat vs Fiber Density — Where Are the Truly Healthy Snacks?",
    height=440,
)
fig_heat.add_annotation(
    x=3, y=heat_df["fiber_100g"].max() * 0.85,
    text="Ideal zone: High Fiber + Low Fat",
    showarrow=False, font=dict(size=11, color="#1a6b35")
)
fig_heat.update_layout(
    plot_bgcolor="white",
    paper_bgcolor="white",
```

```
    font=dict(family="Arial", size=12),
)
st.plotly_chart(fig_heat, use_container_width=True)
st.markdown("---")

# STORY 4 — KEY MARKET INSIGHT & RECOMMENDATION
# "Based on the data, the biggest market opportunity is in
# [Category], specifically targeting products with [X]g of protein
# and less than [Y]g of sugar."

st.subheader("Story 4 — Key Market Insight & Recommendation")

best_cat = (
    df.groupby("primary_category")
    .apply(lambda g: int(((g["proteins_100g"] > 10) & (g["sugars_100g"] < 5)).sum()))
    .idxmax()
)
best_grp = df[
    (df["primary_category"] == best_cat) &
    (df["proteins_100g"] > 10) &
    (df["sugars_100g"]   < 5)
]
avg_prot = best_grp["proteins_100g"].mean()
avg_sug  = best_grp["sugars_100g"].mean()

st.success(
    f"Based on the data, the biggest market opportunity is in **{best_cat}**, "
    f"specifically targeting products with **{avg_prot:.0f}g of protein** "
    f"and less than **{avg_sug:.0f}g of sugar** per 100g.\\n\\n"
    f"Across 500,000 products analysed, only **{len(best_grp):,}** currently "
    f"occupy this space — a clear, underserved gap that represents a strong "
    f"first-mover opportunity for our client."
)
st.markdown("---")

# BONUS STORY — THE HIDDEN GEM: TOP PROTEIN SOURCES
# Analyse ingredients_text in the High Protein / Low Sugar cluster
# to identify the top 3 protein sources for the R&D team.
st.subheader("Bonus Story — The Hidden Gem: Top Protein Sources")
st.caption(
    "To help the R&D team formulate the new product, we analysed the "
    "ingredient lists of every High-Protein (>10g) / Low-Sugar (<5g) "
    "product to find the most common protein sources driving success "
    "in the Blue Ocean zone."
)

hp_df = df[
    (df["proteins_100g"] > 10) &
    (df["sugars_100g"]   < 5)  &
    df["ingredients_text"].notna()
]

PROTEIN_SOURCES = [
    "whey", "casein", "soy", "pea protein", "egg", "peanut", "almond",
    "milk protein", "hemp", "rice protein", "chickpea", "lentil",
    "quinoa", "sunflower seed", "pumpkin seed",
]

counts = Counter()
for text in hp_df["ingredients_text"].dropna():
    t = text.lower()
    for src in PROTEIN_SOURCES:
        if src in t:
            counts[src] += 1

top10 = counts.most_common(10)

if top10:
    src_df = pd.DataFrame(top10, columns=["Ingredient", "Count"])
    fig_ing = px.bar(
        src_df,
        x="Count",
        y="Ingredient",
        orientation="h",
        color="Count",
        color_continuous_scale="Teal",
        title="Most Common Protein Sources in High-Protein / Low-Sugar Products",
        height=400,
    )
    fig_ing.update_layout(
        yaxis=dict(autorange="reversed"),
        plot_bgcolor="white",
```

```
        paper_bgcolor="white",
        font=dict(family="Arial", size=12),
    )
    st.plotly_chart(fig_ing, use_container_width=True)

    top3 = ", ".join([x[0].title() for x in top10[:3]])
    st.info(
        f"**Top 3 protein sources:** {top3}\\n\\n"
        f"R&D recommendation: Lead formulation with these ingredients "
        f"to deliver a credible high-protein claim with a clean label."
    )
else:
    st.warning(
        "Not enough ingredient data for this filter. "
        "Try widening your sidebar filters."
    )

st.markdown("---")
st.caption(
    "Data: Open Food Facts · openfoodfacts.org · First 500,000 rows · "
    "Analysis by Helix CPG Partners"
)
'''

with open("app.py", "w") as f:
    f.write(APP_CODE)

print("app.py written successfully")
```

```
app.py written successfully
```

requirements.txt for streamlit cloud

```
REQUIREMENTS = """streamlit
pandas
plotly
pyarrow
"""

with open("requirements.txt", "w") as f:
    f.write(REQUIREMENTS)

print("requirements.txt created")
```

```
requirements.txt created
```

Github Deploy

```
import os, shutil

# ── Only paste your token here ──────────────────────────────
GITHUB_TOKEN = "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"

# ── Your details — already filled in ────────────────────────
GITHUB_USERNAME = "ArthurAbigail"
REPO_NAME       = "The-Market-Gap-Analysis"
YOUR_EMAIL      = "arthurabigail203@gmail.com"
YOUR_NAME       = "Arthur Abigail"

# ── Clean up ────────────────────────────────────────────────
os.chdir("/content")
shutil.rmtree("deploy", ignore_errors=True)
os.makedirs("deploy", exist_ok=True)

# ── Copy files ──────────────────────────────────────────────
for f in ["app.py", "requirements.txt",
          "cleaned_food_data.parquet",
          "opportunity_scores.parquet"]:
    shutil.copy(f, f"deploy/{f}")
    print(f"  copied: {f}")

os.chdir("/content/deploy")

# ── Git config ──────────────────────────────────────────────
os.system('git config --global user.email "arthurabigail203@gmail.com"')
os.system('git config --global user.name "Arthur Abigail"')
os.system('git config --global init.defaultBranch main')

# ── Remote URL ──────────────────────────────────────────────
```

```
REMOTE_URL = "https://" + GITHUB_USERNAME + ":" + GITHUB_TOKEN + "@github.com/" + GITHUB_USERNAME + "/" + REPO_NAME + ".gi

print("Pushing to: https://github.com/ArthurAbigail/The-Market-Gap-Analysis")

# ── Push ──────────────────────────────────────────────────────────
os.system("git init")
os.system("git add .")
os.system('git commit -m "Add Sugar Trap dashboard — Stories 1-4, Bonus, Candidates Choice"')
os.system("git remote add origin " + REMOTE_URL)
os.system("git branch -M main")
os.system("git push -u origin main --force")

print("\n✅ Done!")
print("Check : https://github.com/ArthurAbigail/The-Market-Gap-Analysis")
print("Next  : https://share.streamlit.io")
```

```
  copied: app.py
  copied: requirements.txt
  copied: cleaned_food_data.parquet
  copied: opportunity_scores.parquet
Pushing to: https://github.com/ArthurAbigail/The-Market-Gap-Analysis

✅ Done!
Check : https://github.com/ArthurAbigail/The-Market-Gap-Analysis
Next  : https://share.streamlit.io
```

Quick Fix

```
import os, shutil

with open("/content/app.py", "r") as f:
    content = f.read()
OLD = """st.dataframe(
    display_df.style.background_gradient(subset=["Score"], cmap="RdYlGn"),
    use_container_width=True,
    hide_index=True,
)"""

NEW = """st.dataframe(
    display_df,
    use_container_width=True,
    hide_index=True,
)"""

content = content.replace(OLD, NEW)

with open("/content/app.py", "w") as f:
    f.write(content)

print("app.py fixed")

shutil.copy("/content/app.py", "/content/deploy/app.py")
print("Copied to deploy folder")

# Push to GitHub
os.chdir("/content/deploy")

GITHUB_TOKEN    = "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"
GITHUB_USERNAME = "ArthurAbigail"
REPO_NAME       = "The-Market-Gap-Analysis"

REMOTE_URL = "https://" + GITHUB_USERNAME + ":" + GITHUB_TOKEN + "@github.com/" + GITHUB_USERNAME + "/" + REPO_NAME + ".git"

os.system("git add .")
os.system('git commit -m "Fix dataframe styling error"')
os.system("git push origin main --force")

print("\n Done! Wait then refresh your Streamlit link.")
```

```
app.py fixed
Copied to deploy folder

 Done! Wait then refresh your Streamlit link.
```